RideFlow Free-Hosting and Database Setup Guide

Purpose: Deploy the portable RideFlow full-stack application using free or low-cost services, connect the server to a hosted database, configure authentication and private file storage, and prepare sandbox payments.

> **Important:** Free tiers are suitable for learning, testing, and a small private pilot. They are not automatically suitable for a public ride-sharing service. Free services may sleep, pause, restart, expire, have no service-level agreement, or lack reliable backups. Do not store real identity documents or process real customer payments until you have reviewed the provider terms and added backups, monitoring, support, and a production plan.

Recommended low-cost architecture

For the current RideFlow codebase, the simplest compatibility-first arrangement is:

Layer	Recommended starting service	Why
Source	Private GitHub repository `raiven427/ride-flow`	Deployment source and version history
Web/API server	Render Free Web Service for testing	Supports Node/Express and GitHub deploys
MySQL-compatible database	Aiven for MySQL Free Tier	Matches RideFlow's `mysql2` and Drizzle MySQL setup
Object storage	Any S3-compatible provider with a free allowance	RideFlow stores file bytes outside the database
Authentication	An OIDC provider with a development/free plan	Passwords remain with the identity provider
Payments	Stripe test mode and Safaricom Daraja sandbox	No real money while integrating
Domain	Provider subdomain initially	Add a custom domain after HTTPS, callbacks, and login work

Render's free web services sleep after 15 minutes without inbound traffic and can take about a minute to wake. Their filesystem is ephemeral, so uploaded files and local SQLite data must not be stored there. Render also states that free services are intended for testing and hobby projects rather than production [1]. Aiven's free MySQL tier is compatible with the current RideFlow database driver and includes a single node, 1 GB RAM, 1 GB disk, monitoring, and backups, but has no 99.99% SLA and can power down inactive services [2].

Supabase is another useful free service, but its free database is PostgreSQL. RideFlow currently uses MySQL/TiDB through `mysql2`; using Supabase would require a deliberate PostgreSQL migration rather than only changing `DATABASE_URL`. Supabase's free plan also pauses projects after one week of inactivity and does not include automatic backups [3].

1. Prepare Ubuntu

Install the basic tools on Ubuntu 22.04 or 24.04:

```
sudo apt update
sudo apt install -y git curl unzip
curl -fsSL https://get.pnpm.io/install.sh | sh -
source ~/.bashrc
node --version
pnpm --version
git --version
```

Clone the private repository:

```
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install --frozen-lockfile
pnpm check
pnpm test
pnpm build
```

If GitHub requests authentication, use GitHub's browser sign-in or a GitHub token through the credential manager. Never place a GitHub token in a source file, shell history, PDF, or screenshot.

2. Create the free MySQL database

Create an Aiven account, create one **Aiven for MySQL** service on the free tier, and choose a region close to your web service. Wait until the service reports that it is running. Create a dedicated database user for RideFlow rather than using a personal administrator account.

Copy the provider's secure connection values into a password manager. The final connection string should have this general shape, with your real values entered only in the host secret manager:

```
mysql://RIDEFLOW_USER:RIDEFLOW_PASSWORD@HOST:PORT/RIDEFLOW_DATABASE
```

Use the exact URI format supplied by the provider. Some hosts include TLS parameters or special characters that must be URL-encoded. Do not guess the port, database name, or TLS options.

Test the connection from Ubuntu without printing the password:

```
export DATABASE_URL='mysql://user:password@host:port/database'
node -e "import('mysql2/promise').then(async ({default:mysql}) => { const
```

If the command fails, check the hostname, port, TLS requirement, allowlist, user permissions, and whether the free database is powered on.

Run the RideFlow migrations only after confirming the connection:

```
pnpm drizzle-kit generate
pnpm drizzle-kit migrate
```

The source schema is in `drizzle/schema.ts`. Do not edit production tables manually without first updating the schema, generating a migration, testing it on a disposable database, and keeping a backup.

3. Create the free web service

Connect the private GitHub repository to the hosting provider. Create a **Web Service**, not a static site, because RideFlow has an Express/tRPC server and database access.

Use these settings as the starting point:

Setting	Value
Repository	`raiven427/ride-flow`
Branch	`main`
Runtime	Node
Build command	`corepack enable && pnpm install --frozen-lockfile && pnpm build`
Start command	`pnpm start`
Region	Same or nearby region as the database
Health check	`/healthz`
Auto-deploy	On push after the first successful manual deployment

Render requires a web service to bind to `0.0.0.0` and the `PORT` environment variable. The application must not hardcode a production port [4]. If the service logs say that no port was detected, inspect `server/_core/index.ts` and confirm the listener uses the provider port and host. The provider's default port may differ from local development.

The first deployment may fail if a dependency is installed with a different package-manager version. Keep the committed `pnpm-lock.yaml`, use the package manager version declared in `package.json`, and read the provider build log before changing the build command.

4. Add environment variables safely

In the host's **Environment Variables** or **Secrets** panel, add the following values. Do not commit a populated `.env` file. The repository contains a template at `docs/deployment-environment.template`.

Core server values

```
NODE_ENV=production
PORT=<provided automatically by the host when supported>
PUBLIC_APP_URL=https://<your-provider-domain>
DATABASE_URL=<Aiven MySQL connection string>
JWT_SECRET=<long random value>
```

Generate a random secret on Ubuntu without exposing it in a command log:

```
openssl rand -base64 48
```

Authentication values

Create an OIDC application with your chosen identity provider. Set its callback URL to the exact production URL required by RideFlow's auth adapter. Configure:

```
OAUTH_ISSUER_URL=<provider issuer>
OAUTH_TOKEN_URL=<provider token endpoint>
OAUTH_USERINFO_URL=<provider userinfo endpoint>
OAUTH_CLIENT_ID=<server/client id>
OAUTH_CLIENT_SECRET=<server secret>
VITE_OAUTH_AUTHORIZE_URL=<browser authorize endpoint>
VITE_OAUTH_CLIENT_ID=<browser client id if required>
VITE_OAUTH_SCOPE=openid profile email
```

Use HTTPS in production. If login loops, verify the redirect URI, cookie security, application URL, provider clock, client ID, and callback route. Passwords belong to the identity provider and must not be added to RideFlow's database or source code.

Private S3-compatible storage

Create a private bucket for profile images and driver documents. Add:

```
S3_BUCKET=<private bucket>
S3_REGION=<provider region>
S3_ENDPOINT=<S3-compatible endpoint>
S3_ACCESS_KEY_ID=<storage access key>
S3_SECRET_ACCESS_KEY=<storage secret>
S3_FORCE_PATH_STYLE=false
S3_PUBLIC_BASE_URL=
S3_SIGNED_URL_TTL_SECONDS=900
S3_SERVER_SIDE_ENCRYPTION=<provider option if supported>
```

Keep the bucket private. RideFlow should return short-lived signed URLs rather than public permanent document URLs. Test with a non-sensitive image first. Confirm that the web service's local filesystem is never used for durable uploads.

Admin and notifications

```
RIDEFLOW_ADMIN_EMAIL=your-admin-email@example.com
NOTIFICATION_PROVIDER_URL=<webhook or email provider endpoint>
NOTIFICATION_PROVIDER_TOKEN=<secret token>
```

For a free pilot, a webhook notification service may be simpler than SMTP. Render free services cannot send outbound SMTP on ports 25, 465, or 587 [1]. Verify that the notification provider accepts HTTPS requests and that the token is stored only as a server secret.

5. Configure Stripe in test mode

Create or select a Stripe test sandbox. Use only test keys while developing:

```
PAYMENTS_STRIPE_ENABLED=true
STRIPE_SECRET_KEY=sk_test_...
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_CONNECT_CLIENT_ID=ca_...
```

Configure the HTTPS webhook URL using the deployed domain and the route documented in `server/payments.ts`. The server must receive the raw body, verify the `Stripe-Signature` header, reject invalid signatures, and handle duplicate event IDs idempotently. Stripe's official documentation recommends signature verification and a fast 2xx response before complex work [5].

Use Stripe's official test cards. Test a successful payment, declined payment, duplicate webhook, invalid webhook signature, and provider timeout. Confirm that the 5% RideFlow commission and driver amount are written once to the append-only ledger. Never mark a trip paid merely because the browser says payment succeeded.

6. Configure M-Pesa Daraja sandbox

Create a Safaricom Daraja developer account and a sandbox app. Add:

```
PAYMENTS_DARAJA_ENABLED=true
DARAJA_ENVIRONMENT=sandbox
DARAJA_CONSUMER_KEY=<sandbox consumer key>
DARAJA_CONSUMER_SECRET=<sandbox consumer secret>
DARAJA_SHORTCODE=<sandbox shortcode>
DARAJA_PASSKEY=<sandbox passkey>
DARAJA_INITIATOR_NAME=<sandbox initiator if required>
DARAJA_SECURITY_CREDENTIAL=<sandbox credential if required>
DARAJA_CALLBACK_BASE_URL=https://<your-provider-domain>/api/webhooks/daraj
```

The Daraja M-Pesa Express simulator uses the sandbox STK Push endpoint and sends the final result asynchronously to the callback URL [6]. The initial response means that the request was accepted; it does not prove that the customer completed payment.

Before testing, confirm the callback URL is public HTTPS and not a local `localhost` URL. Submit a sandbox STK request, store the MerchantRequestID and CheckoutRequestID as pending, then wait for the callback. Mark the payment successful only when the callback result code indicates success and the callback identifiers match the pending request. Make the callback idempotent so repeated callbacks cannot duplicate a fare, commission, or driver payout.

7. DNS and HTTPS

Start with the free provider subdomain. Once the application works, add a custom domain through the host's domain panel. At the DNS provider, create the exact records shown by the

host. Do not invent an A record when the host requires a CNAME or verification record.

After DNS propagates, test:

```
curl -I https://<your-domain>/healthz
curl -I https://<your-domain>/
```

Then update `PUBLIC_APP_URL`, OIDC redirect URIs, Stripe webhook URL, Daraja callback URL, and any notification webhook allowlist. A callback configured for the old provider subdomain will fail after a domain change.

8. Backups and free-tier risks

Free-tier services are appropriate for a prototype only when you can recreate them. Export the database before migrations and on a regular schedule. Store encrypted backups in a different provider or offline location. Test restoration, not only backup creation.

Aiven’s free MySQL tier advertises backups but has no production SLA and may power down an inactive service [2]. Render free web services have ephemeral filesystems and free database offerings may expire [1]. Supabase free projects pause after inactivity and the free plan does not include automatic backups [3]. These limits are why a public ride-sharing launch should move to paid, monitored services before real users depend on it.

9. Verification checklist

Check	Expected result
`GET /healthz`	HTTP 200 and no secret values in the response
Landing page	Loads over HTTPS
OIDC login	Returns to RideFlow without a redirect loop
Customer quote	Server-side quote and ledger rows are created
Driver upload	Authenticated upload is stored in private object storage
Admin operations	Only the admin can read the operations snapshot
Stripe sandbox	Test event is signed, accepted, and idempotent
Daraja sandbox	STK request becomes pending, then callback resolves it
Database migration	All expected tables exist and application queries work
Restart	App recovers without losing database or storage data
Free-tier sleep	Wake-up behavior is understood and acceptable for testing
Backup restore	A recent backup can recreate the database

10. Troubleshooting

A deployment that exits immediately usually has a build/start command, Node version, missing dependency, or environment-variable problem. A service that starts but cannot receive traffic usually binds to the wrong host or port. A login loop usually comes from an incorrect callback URL or cookie configuration. A database connection error usually comes from an incorrect URI, TLS requirement, firewall/allowlist, powered-off free database, or insufficient user permission.

If uploads disappear after a restart, the application is writing to local disk instead of S3-compatible storage. If Stripe or Daraja callbacks remain pending, first check that the callback URL is public HTTPS, then inspect provider delivery logs and RideFlow server logs. If the free service sleeps, the first request may be slow; do not add artificial traffic to bypass provider limits.

11. What to change before real launch

Move the server and database to plans with a documented SLA, automatic backups, monitoring, and support. Add a separate realtime service for GPS and dispatch. Add rate limiting, incident alerts, payment reconciliation, refund/dispute procedures, driver verification, data retention, access reviews, and a tested restore procedure. Review Kenya-specific transport, privacy, payment, and employment obligations with qualified local advisers.

References

[1]: https://render.com/docs/free Render, “Deploy for Free.”

[2]: https://aiven.io/docs/products/mysql/concepts/mysql-free-tier Aiven, “Aiven for MySQL free tier.”

[3]: https://supabase.com/pricing Supabase, “Pricing & Fees.”

[4]: https://render.com/docs/web-services Render, “Web Services.”

[5]: https://docs.stripe.com/webhooks Stripe, “Receive Stripe events in your webhook endpoint.”

[6]: https://developer.safaricom.co.ke/apis/MpesaExpressSimulate Safaricom Daraja, “M-Pesa Express Simulate.”